Mortgage Document Package Processor

Software Engineering Intern — Architecture Case Study

Stage 1: Ingestion & Page Splitting

Objective

Normalize the raw 2,000-page PDF into discrete, individually addressable page units, and tag each page's rendering modality (digital vs. scanned) so downstream stages know exactly what processing is needed.

Logic & Algorithm

- Load the PDF using PyMuPDF (fitz) and extract every page as a standalone in-memory object.
- Attempt text extraction on each page via `page.get_text()`. If the returned string contains more than 50 meaningful (non-whitespace) characters, tag the page as *DIGITAL*; otherwise tag it as *SCANNED*.
- Rasterize all *SCANNED* pages to 300 DPI PNG images for downstream *OCR*. *DIGITAL* pages skip rasterization entirely, saving time and compute.
- Compute a perceptual hash (pHash) of each page image immediately and store it in a global hash map. This is used later in Stage 7 for duplicate detection.
- Assign each page a unique page_id (package_id + 0-indexed position). Persist metadata: page_id, position, modality, image_path, and raw_text.

```
{
  "page_id": "pkg_001_0042",
  "package_id": "pkg_001",
  "position": 42,
  "modality": "SCANNED",
  "image_path":
"s3://mortgage-packages/pkg_001/pages/page_0042.png",
  "raw_text": "Uniform Residential Loan
Application\nBorrower Name: John Smith\nDate:
03/15/2024\nPage 3 of 8"
}
```

Tools & Technologies

- PyMuPDF (fitz) — PDF loading and native text extraction
- pdf2image / Pillow — rasterizing scanned pages to PNG
- imagehash — perceptual hashing for duplicate detection
- Python multiprocessing.Pool — parallel processing (splitting is I/O-bound, rasterization is CPU-bound; use separate pools)

- AWS S3 / Google Cloud Storage — storing page images and metadata

Cost & Latency Control

No LLM calls at this stage. PyMuPDF is a free, pure-Python library. Rasterization takes approximately 0.5–1 second per scanned page on a modern CPU core. Parallelizing across cores makes this stage fast and essentially free. Investing compute here reduces expensive LLM calls downstream.

Stage 2: OCR & Text Extraction

Objective

Produce a machine-readable text layer for every page in the package. DIGITAL pages already have text from Stage 1. This stage processes only SCANNED pages using OCR engines that also return structured output (bounding boxes, tables, form fields).

Logic & Algorithm

- Tiered OCR strategy: First run Tesseract 5 locally (free, fast). If Tesseract returns a confidence score below 70%, escalate the page to AWS Textract.
- AWS Textract's AnalyzeDocument API is used with FORMS and TABLES features enabled. This returns structured key-value pairs and table cell data, which is critical for extracting fields from the 1003 loan application form.
- Store a structured object per page: raw_text, text blocks with bounding boxes, tables as row/column arrays, and key-value pairs from form fields.
- Pre-process scanned images before OCR: deskew using OpenCV's Hough line transform, binarize using Otsu thresholding, and remove page borders. This can improve Tesseract accuracy by 10–20%.

Tools & Technologies

- Tesseract 5 — free, local OCR for the majority of scanned pages
- AWS Textract — cloud OCR with form and table understanding for difficult pages
- OpenCV — image pre-processing (deskew, binarize, denoise)
- pytesseract — Python wrapper for Tesseract
- boto3 — AWS SDK for calling Textract

Cost & Latency Control

Tesseract is free and processes a page in roughly 0.3 seconds. Textract escalation is gated on Tesseract confidence — only genuinely difficult pages incur cloud cost. Textract costs approximately $0.015 per page for FORMS analysis. In a 2,000-page package with ~40% scanned pages (800 pages) and ~15% escalation rate, the Textract cost is roughly $1.80 per package. Compare this to running GPT-4o Vision on all 2,000 pages at $0.01–0.02 per page, which would cost $20–$40 per package. The OCR-first approach is 10–20 times cheaper.

Stage 3: Feature Engineering

Objective

Extract lightweight classification signals from each page's text — keywords, form identifiers, page numbers, date patterns, and layout features — to build a feature vector that drives the rule-based classifier in Stage 4 without any LLM calls.

Logic & Algorithm

- Header and title keywords: Use regex to detect phrases such as 'Uniform Residential Loan Application', 'Form 1003', 'W-2 Wage and Tax Statement', 'Form 1040', 'Credit Report', 'Appraisal Report', 'Title Insurance', and 'HUD-1'.
- Structural form markers: Look for Fannie Mae and Freddie Mac form numbers, SSN patterns (xxx-xx-xxxx), employer identification numbers (EIN), loan numbers, and ABA bank routing numbers.
- Page number extraction: Use regex to find patterns such as 'Page X of Y', 'Page X', and '- X -'. These are critical for sequencing pages within a document type later.
- Date patterns: Extract all dates in common formats (MM/DD/YYYY, Month YYYY). Bank statements have consistent statement periods; tax returns have tax years. These anchor document identity.
- Table density score: Calculate the ratio of tabular content area (from Textract output) to total page area. A high density score typically indicates a financial document like a bank statement or 1040.
- Signature zone detection: Look for 'Signature' or 'Sign Here' text labels, underline patterns near the bottom of a page, and adjacent date fields. Record whether each zone appears filled or empty.

Tools & Technologies

- Python re (regex) — pattern matching for keywords, form numbers, dates, page numbers
- spaCy — named entity recognition for names, organizations, and dates
- dateparser — robust date extraction across multiple formats
- numpy — feature vector construction and scoring

Cost & Latency Control

This stage uses only Python libraries — no API calls of any kind. Processing time is under 5 milliseconds per page, so the full 2,000 pages complete in roughly 10 seconds. The richer the signals extracted here, the fewer pages slip through to the expensive LLM gate in Stage 5.

Stage 4: Rule-Based Page Classification

Objective

Classify the large majority of pages into their correct document types using deterministic rules and confidence scoring — with zero LLM cost. Only pages that fall below the confidence threshold are passed to Stage 5 for LLM review.

Logic & Algorithm

- Each document type has a corresponding rule that checks for specific combinations of signals from the Stage 3 feature vector. For example: a page with keyword 'Uniform Residential Loan Application', no filled signature zone, and a page number → classified as LOAN_APP_INITIAL with 0.95 confidence.
- Confidence scores are computed by weighting matched signals. Strong signals (official form number match) contribute more weight than weak signals (generic financial keywords).
- Pages scoring above 0.85 confidence are immediately classified and routed to Stage 6. Pages between 0.60 and 0.85 receive a tentative label and are flagged for LLM verification. Pages below 0.60 are sent to Stage 5 with no label.
- In practice, rule-based classification handles approximately 85% of all pages in a typical loan package.

Classification Rules (Examples)

- LOAN_APP_INITIAL: keyword 'Uniform Residential Loan Application' + no signature in signature zone → confidence 0.95+
- LOAN_APP_FINAL: same keyword + signature zone filled + date present → confidence 0.92+
- BANK_STATEMENT: 'Statement Period' or 'Account Summary' + table density > 0.5 + routing or account number → confidence 0.90+
- W2: 'Wages, tips, other compensation' + form number 'W-2' + employer EIN → confidence 0.98+
- TAX_1040: 'U.S. Individual Income Tax Return' + 'Form 1040' → confidence 0.97+
- PAY_STUB: 'Pay Period', 'Gross Pay', 'Net Pay', 'YTD' all present on same page → confidence 0.90+
- CREDIT_REPORT: 'FICO', 'Credit Score', 'Tradelines', any of Equifax/Experian/TransUnion → confidence 0.93+
- APPRAISAL: 'Uniform Residential Appraisal Report', 'URAR', 'Market Value' → confidence 0.95+

Cost & Latency Control

Zero LLM calls. Pure Python logic running at millisecond speed. The goal is to make this gate as effective as possible — every additional page correctly classified here eliminates one page from the LLM cost in Stage 5.

Stage 5: LLM-Assisted Classification (Fallback Gate)

Objective

Use an LLM only for the small subset of pages that the rule-based classifier could not confidently classify. The LLM is a fallback gate of last resort, not a general-purpose classifier for the whole package.

Logic & Algorithm

- Text-only prompting: For pages where OCR produced more than 100 usable characters, send the extracted text to the LLM — never the image. Text tokens are approximately 10 times cheaper than vision tokens.
- Batching: Group up to 20 ambiguous pages into a single LLM API call. The prompt asks the model to classify each page and return a JSON array with page_id, document type, and confidence. One API call handles 20 pages.
- Vision fallback: Pages with fewer than 100 OCR characters (genuinely unreadable scans) are sent as images at reduced resolution (150 DPI) to a vision-capable model (Claude claude-sonnet-4-20250514 or GPT-4o). These are the most expensive calls and typically represent only 2–5% of all pages.
- Structured output enforcement: JSON mode or function calling is used to force the model to return machine-parseable output. No prose, no explanations — just the classification array.
- Short system prompt: The system prompt is kept under 200 tokens. It contains only the list of valid document types and the output schema. The model already has general knowledge of what a W-2 or bank statement looks like.

Tools & Technologies

- GPT-4o — for text-based classification
- GPT-4o Vision — for image-based fallback on unreadable pages
- JSON mode / function calling — for structured output
- Python asyncio — for batching and concurrent API calls

Cost & Latency Control

Approximately 15% of 2,000 pages = 300 pages reach this stage. Batched into groups of 20, that is 15 API calls total. Text-only calls (roughly 80% of the 300) cost about $0.003 each. Vision calls (roughly 20%) cost about $0.05 each. Total LLM cost per package is approximately $0.19. At 1,000 packages per day, total daily LLM spend is approximately $190 — a bounded and predictable cost.

Stage 6: Document Assembly & Sequencing

Objective

Group all classified pages by document type, split groups into distinct document instances where multiple copies exist, and order the pages within each document correctly.

Grouping Logic

- Primary grouping: All pages with the same document type label are collected into one initial group (e.g., all BANK_STATEMENT pages).
- Multi-instance detection: Within a group, use secondary signals to identify distinct document instances. Bank statements are split by statement period (e.g., January 2024 vs. February 2024). Tax returns are split by tax year. Form 1003 pages are split by initial vs. final flag and by signer name if multiple borrowers exist.
- Entity linking: Link each bank statement instance to a specific borrower via name and account number matching. Confirm that each 1040 tax return's SSN matches the SSN on the 1003 loan application. Flag any mismatches as exceptions.

Page Sequencing Logic

- Explicit page numbers: If a page contains 'Page X of Y', sort all pages in the group by the extracted X value in ascending order.
- Statement date continuity: For bank statements, order pages by the flow of transaction dates (earliest transactions first, then later ones).
- Form section flow: The 1003 form has a known section order (Section I through Section VIII). Match detected section headers to enforce the correct ordering.
- Topological fallback: If no explicit ordering signal is available, use the original position of the page within the raw PDF as a proxy — pages that were originally adjacent are likely from the same sequential run.

Cost & Latency Control

Zero LLM calls. All logic is deterministic Python. Grouping and sorting 2,000 classified pages completes in well under one second.

Stage 7: Validation & Quality Assurance

Objective

Detect and flag anomalies in the assembled documents: missing required documents, missing pages within a document, unsigned documents that require signatures, and duplicate pages. Generate structured exception records rather than crashing — the system always produces partial output.

Validation Checks

- Required document checklist: Assert that each required document type is present in the package: LOAN_APP_FINAL, BANK_STATEMENT (3 months), W2, TAX_1040, CREDIT_REPORT, APPRAISAL, and TITLE_DOC. Flag any that are absent.
- Page count validation: If a page contained 'Page X of Y', assert that the assembled group contains exactly Y pages. If the group has 5 pages but the form says 'Page 1 of 8', generate a MISSING_PAGES exception.
- Signature detection: For documents that legally require a signature (1003 Final, disclosure forms), assert that at least one signature zone has been detected as filled. Use OpenCV to detect ink strokes in the signature bounding boxes identified in Stage 3. Flag unsigned required documents as MISSING_SIGNATURE.
- Duplicate detection: Compare the perceptual hash (pHash) values computed in Stage 1. Any two pages with a Hamming distance of 4 or less are considered near-duplicates. Flag and exclude duplicates from the final assembled output.
- Version conflict detection: If both LOAN_APP_INITIAL and LOAN_APP_FINAL exist and share identical dates, flag as VERSION_AMBIGUOUS for human review.
- Date range coverage: For bank statements, verify that there are no missing months in the required coverage window (typically the last 3 months). Flag any gap as MISSING_STATEMENT_PERIOD.

Exception Severity Levels

- CRITICAL — Examples: no signed 1003 final found, no credit report present. Action: block decisioning, require re-submission of the package.
- WARNING — Examples: one month of bank statements is missing, duplicate pages detected. Action: flag for underwriter review, do not block processing.
- INFO — Examples: initial 1003 is present alongside the final version. Action: log the observation, use the final version, note that both exist.
- OK — All checks passed. Proceed to output generation.

Tools & Technologies

- OpenCV — signature ink stroke detection in scanned signature zones
- imagehash — perceptual hash comparison for duplicate detection
- Pydantic — schema validation for exception records and output objects

Cost & Latency Control

Zero LLM calls. OpenCV signature detection runs in milliseconds per page. Hash comparison is O(n) over 2,000 hashes and completes instantly. This stage is fast and free.

Stage 8: Output Generation & Exception Reporting

Objective

Emit a structured JSON manifest for the downstream decisioning system, reconstituted per-document PDF files in the correct page order, and a human-readable exception report summarizing all issues found.

Output Artifacts

- Structured JSON manifest: One JSON object per document instance containing its type, ordered list of page numbers, extracted metadata (dates, names, amounts), validation status, and any exception codes.
- Reconstituted PDFs: Re-assemble per-document PDF files using the ordered page list. Write to blob storage using a deterministic path: s3://output/{package_id}/{doc_type}/{instance_id}.pdf
- Exception report: A JSON and human-readable HTML summary of all WARNING and CRITICAL exceptions, including the affected page IDs, exception codes, and recommended actions for the underwriter.
- Processing metadata: Pipeline latency per stage, total LLM call count, OCR escalation count, page counts by modality, and total cost. Used for SLA monitoring and cost tracking.

Tools & Technologies

- PyMuPDF — re-assembling reconstituted per-document PDFs
- Pydantic — validating output JSON schema before writing
- AWS S3 / Google Cloud Storage — storing output PDFs and manifests
- Jinja2 — rendering the human-readable HTML exception report

Cost & Latency Control

Zero LLM calls. PDF reassembly is fast I/O. Writing JSON is negligible. The entire output stage completes in a few seconds even for a 2,000-page package.

How the Design Addresses Each Constraint

File naming is unreliable

The pipeline ignores filenames entirely. Classification is driven purely by the content of each page — keywords, form numbers, layout patterns, and structural signals — extracted in Stages 3 through 5.

Not all pages are machine-readable

Stage 1 detects each page's modality. Scanned pages are routed through a tiered OCR pipeline (Tesseract first, then AWS Textract for difficult pages). Vision LLM classification in Stage 5 is the final fallback for pages where OCR fails entirely.

LLM calls are expensive

Stages 1 through 4 and Stages 6 through 8 make zero LLM calls. The LLM is engaged only in Stage 5, and only for pages that a deterministic rule engine could not classify with sufficient confidence. Batching (20 pages per call), text-first prompting, and vision-only-when-necessary reduce the total LLM cost to approximately $0.19 per package.

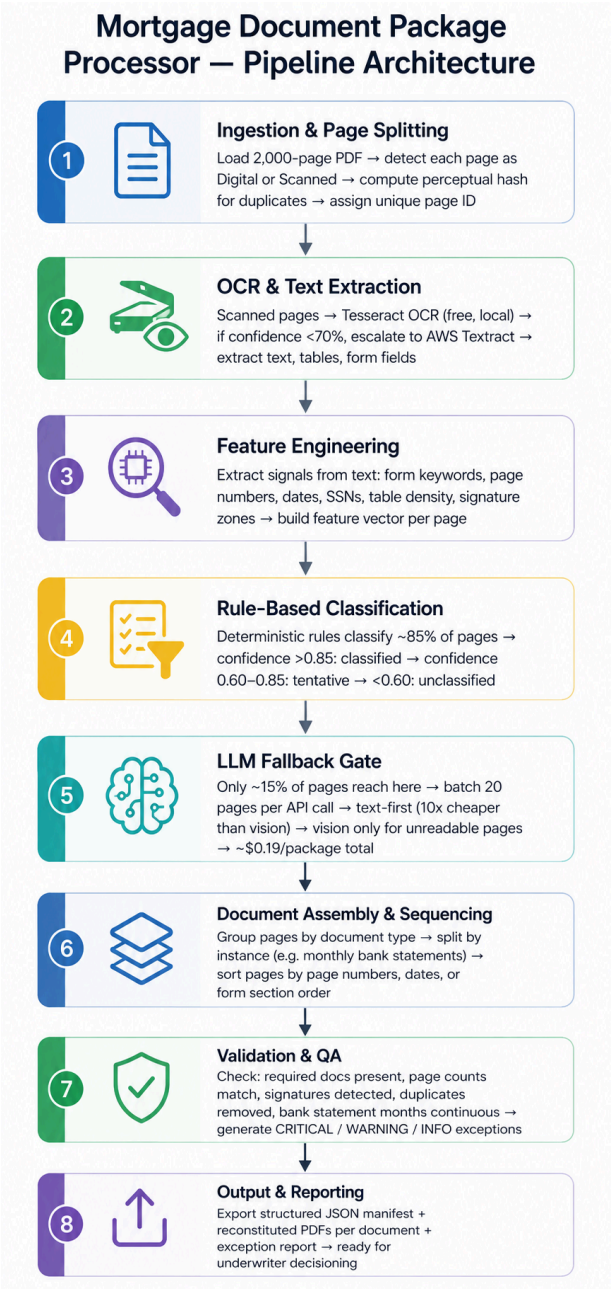
The system must handle packages it has never seen before

The classification rules are anchored to universal, stable document standards: IRS form numbers, Fannie Mae and Freddie Mac form titles, FICO credit bureau terminology. These do not change based on the specific lender or loan type. For genuinely novel pages, the LLM gate handles them gracefully without requiring any retraining.

The system must degrade gracefully

Every anomaly generates a structured exception record (CRITICAL, WARNING, or INFO) rather than throwing an unhandled error. The pipeline stages are independent: Stage 6 assembly runs on whatever pages were successfully classified, even if Stage 5 partially failed. The system always produces partial output plus a complete exception report — never a silent crash.

Performance & Cost Summary

Metric	Target / Estimate
End-to-end processing time per package	~2 minutes
Pages classified without LLM	~85% (1,700 of 2,000 pages)
Pages requiring LLM classification	~15% (300 pages)
LLM API calls per package	~15 batched calls
Estimated LLM cost per package	~$0.19
Daily LLM cost at 1,000 packages	~$190
Daily OCR (Textract) cost at 1,000 packages	~$1,500–$2,250

Metric	Target / Estimate
Packages per day capacity (8–16 CPU cores)	1,000+